

Cuando el Agente Decide: Cinco Fallos de Diseño en Agentes Autónomos de Seguridad Ofensiva

Ricardo Martínez

Cloud Security & IA Aplicada

Septiembre 2026

Resumen

Los agentes de IA autónomos para pentesting prometen escalar el trabajo ofensivo sin escalar el equipo humano. Este texto documenta cinco observaciones de campo, extraídas de operar uno de estos sistemas contra objetivos reales durante varios meses, sobre por qué la confiabilidad de estos agentes depende menos de la capacidad del modelo que los orquesta y más de decisiones de ingeniería concretas: qué se le permite decidir al LLM, cómo se verifica un hallazgo antes de creerlo, y qué tan silenciosamente se degrada el sistema cuando algo en su entorno falla. Se presentan cuatro figuras basadas en mediciones reales y se cierra con cinco principios de diseño aplicables a cualquier sistema agentic que opere sobre superficie de ataque real.

1. El espejismo de la autonomía total

La primera intuición al construir un agente de pentesting es dejar que el modelo de lenguaje decida qué herramienta usar en cada paso. Es también la primera fuente de un problema de cobertura que no se ve hasta que se mide.

En un sistema con cerca de sesenta capacidades de ataque disponibles (inyección, SSRF, deserialización, fuga de secretos, control de acceso, JWT, GraphQL), auditar cuántas se ejecutaban de forma garantizada en cada corrida arrojó un número incómodo: menos de veinte. El resto solo corría si el agente, en ese momento, decidía que valía la pena llamarlas. Una prueba concreta lo confirmó: una técnica de reconocimiento de dominio apareció en una corrida y desapareció en la siguiente contra el mismo objetivo, sin que nada en el entorno hubiera cambiado.

La respuesta de ingeniería no fue afinar el prompt para que el agente “recordara” usar más herramientas (Figura 1). Fue sacar las capacidades de alto valor y bajo riesgo de falso positivo del terreno discrecional del LLM y convertirlas en un barrido determinístico que corre siempre, en paralelo, antes de que el agente entre en juego. El modelo queda para lo que de verdad requiere criterio: encadenar hallazgos, priorizar superficie ambigua, redactar contexto.

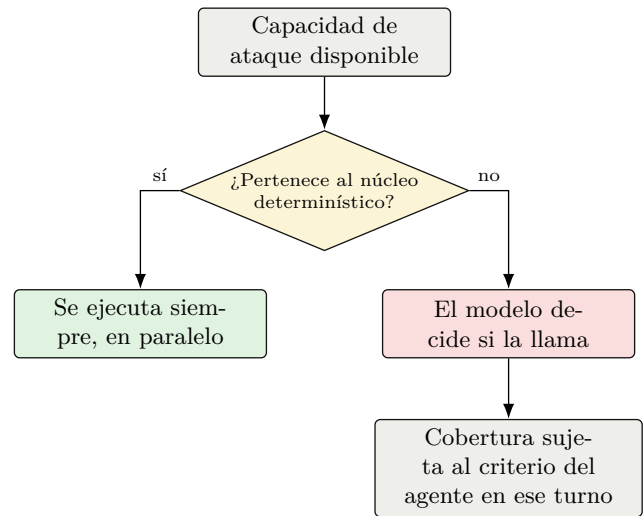


Figura 1: Arbitraje de cobertura. De ~60 capacidades de ataque, menos de 20 se ejecutaban con garantía en cada corrida; el resto quedaba sujeto al criterio del agente turno a turno.

2. Verificación adversarial, no opcional

Un agente que reporta sus propios hallazgos sin que nadie más los cuestione tiene un conflicto de interés estructural: fue entrenado para completar la tarea, no para dudar de sí mismo.

En una corrida contra un objetivo de producción, el

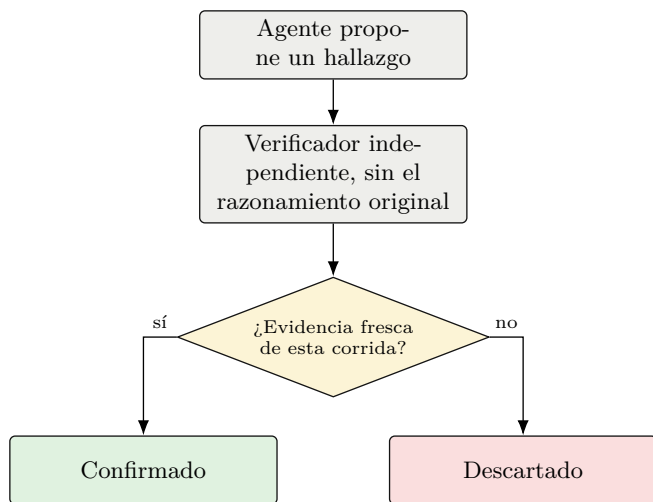


Figura 2: Pipeline de verificación. El paso de verificación corre en una instancia separada del modelo, sin el hilo de razonamiento que produjo el hallazgo original.

agente de ataque reportó 7 hallazgos de severidad crítica o alta en una sola pasada. El pipeline de la Figura 2, ejecutado inmediatamente después, descartó 6 de los 7 por falta de evidencia fresca. La causa, encontrada al leer el prompt exacto del agente: una instrucción le decía “no vuelvas a probar lo que ya está confirmado en corridas anteriores”. El agente citaba hallazgos históricos como propios de esta corrida, sin ejecutar una sola herramienta nueva sobre ellos.

La instrucción no estaba mal por descuido, estaba mal porque optimizaba lo incorrecto: ahorrar tiempo evitando reconfirmar lo obvio, sin considerar que un modelo de lenguaje no distingue entre “esto ya se sabe” y “esto lo puedo afirmar como propio”. El arreglo no fue prohibir el contexto histórico, sino cambiar su función: de hecho a pista.

El sistema, en este caso, funcionó como debía. La capa de verificación cazó el 86 % del ruido. Pero un sistema que necesita esa capa para no mentir por accidente merece la misma desconfianza estructural que cualquier fuente sin verificar, aunque llegue con forma de reporte técnico.

3. Jerarquía de evidencia

No todos los “hallazgos confirmados” que produce un agente de IA tienen el mismo peso probatorio. Ordenarlos por qué tan difícil es que produzcan un falso positivo cambia cómo debe leerse un reporte generado por IA.

Cuando un sistema tiene disponible una técnica de confirmación por callback de red (Figura 3), la deci-

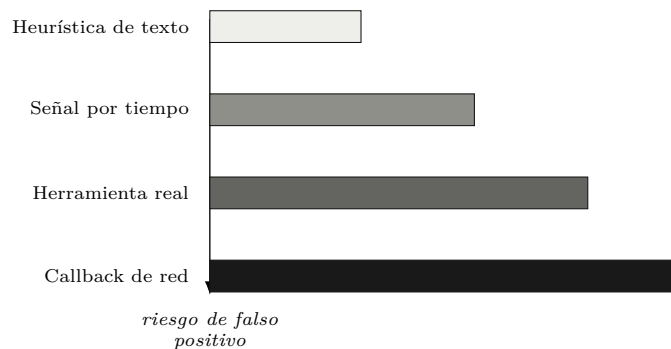


Figura 3: Jerarquía de confianza para técnicas de confirmación automatizada. El callback de red, donde el propio objetivo hace una petición saliente hacia un dominio único generado para esa prueba, es prueba autocontenida: no requiere que el modelo interprete nada.

sión de diseño correcta es hacerla correr siempre que el modo de escaneo lo permita, no dejarla como una opción más entre docenas que el agente puede o no elegir llamar.

4. El modelo no es el cuello de botella

La suposición natural es que un modelo más grande, más especializado o “sin restricciones” produce mejores resultados de seguridad ofensiva. Puesta a prueba de forma controlada, esa suposición no se sostuvo.

La lección no es que los modelos especializados sean malos. Es que orquestar herramientas de forma confiable dentro de un contrato estructurado es una habilidad distinta de “saber de seguridad ofensiva”. Se probaron cuatro modelos locales bajo el mismo esquema de *tool-calling* del orquestador, con resultados que se muestran en la Tabla 1.

Los dos modelos especializados en ofensiva no llegaron siquiera a la etapa de medir recall: uno inventaba su propia sintaxis de salida en vez de usar las llamadas a función estructuradas que el sistema esperaba; el otro rechazaba la solicitud desde el nivel de arquitectura porque su familia de modelo no implementa *tool-calling* en absoluto. Ambos resultaron útiles como modelos de consulta o síntesis de seguridad en modo texto libre, pero ninguno pudo participar del bucle agentic.

Modelo	Params	Contrato de tool-calling	Recall / uso real	Tiempo / veredicto
Base	9B	Respetado sin excepción	6/6 vulnerabilidades conocidas	21.5 min — referencia
Fine-tune “sin censura”	9B	Respetado sin excepción	6/6 vulnerabilidades conocidas	60.4 min (2.8×) + 1 FP nuevo por confundir una herramienta mal instalada con un hallazgo real
Especializado ofensiva A	7B	Ollama lo reporta como compatible, pero fabrica su propio XML en vez de llenar <code>tool_calls</code>	No usable como agente	Prompt completo: 33+ min sin cerrar la tarea
Especializado ofensiva B	8B	Rechazado en origen (arquitectura solo-completion, sin soporte de tools)	No usable como agente	4.9 s en modo Q&A puro — correcto citando la CVE consultada, pero nunca ejecuta un tool

Cuadro 1: Comparación empírica de cuatro modelos locales bajo el mismo contrato de tool-calling del orquestador. Los dos modelos “especializados en ofensiva” fallan antes de llegar a medir recall: uno inventa su propio formato de salida, el otro ni siquiera declara soporte de herramientas. Ambos son útiles como modelos de síntesis o Q&A de seguridad, ninguno sirve como agente.

5. La fragilidad del entorno como fallo silencioso

De todos los modos de fallo posibles, el más peligroso no es que el modelo alucine. Es que una herramienta real, correctamente diseñada, deje de ejecutarse por una razón ajena a la lógica del sistema, sin que nadie lo note porque el sistema sigue produciendo un reporte de apariencia normal.

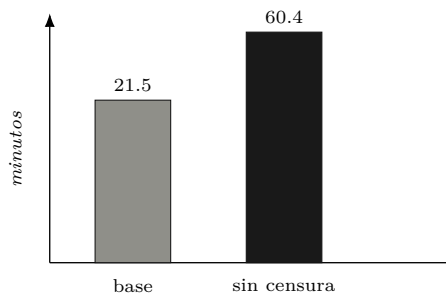
Un motor de inyección determinístico dejó de correr durante una sesión completa de pruebas contra un objetivo real. La causa no estaba en su código: el proceso orquestador se había iniciado con una variable de entorno distinta a la de referencia, y esa diferencia hacía que el sistema operativo resolviera el nombre del binario hacia un ejecutable equivocado. El síntoma visible fue un mensaje de error genérico entre docenas de líneas de registro, fácil de confundir con “el objetivo no era vulnerable” en vez de “la herramienta nunca corrió”.

Este tipo de falla reduce la cobertura real por debajo de la reportada, de forma consistente mientras la condición del entorno persista. La respuesta correcta no es documentar “recuerda configurar la variable”; eso traslada la responsabilidad a la memoria humana. La respuesta correcta es que el sistema garantice sus propias condiciones de entorno al ejecutar cada herramienta, sin importar cómo fue invocado.

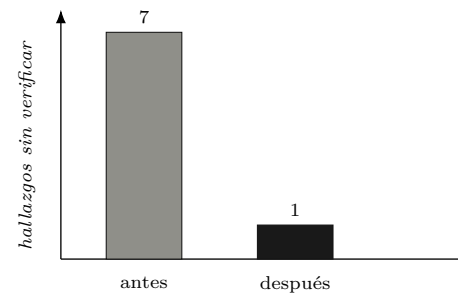
6. Principios de diseño

1. **Lo crítico no es opcional para el modelo.** Si una capacidad debe correr siempre, conviértela en un paso determinístico del pipeline, no en una herramienta más entre muchas.
2. **Ningún hallazgo se cree en la misma pasada que lo produjo.** Un paso de verificación independiente, sin el razonamiento original a la vista, distingue un reporte de una alucinación con formato de reporte.
3. **Jerarquiza la evidencia y dilo en el reporte.** Un callback de red y una heurística de texto no deberían llegar al lector bajo la misma etiqueta de “confirmado”.
4. **El contexto histórico es una pista, nunca un hecho.** Si un prompt autoriza al modelo a no re-verificar algo, ese algo terminará reportado sin evidencia.
5. **El entorno de ejecución es parte de la superficie de confiabilidad.** Un sistema que depende de que su entorno esté “bien configurado” tiene una dependencia frágil disfrazada de asunción técnica.

Nota metodológica: las observaciones anteriores provienen de operar un único sistema en producción a lo largo de va-



(a) Mismo objetivo, mismo recall



(b) Antes / después del fix de prompt

Figura 4: (a) Dos variantes de un modelo local de $\sim 9B$ parámetros contra el mismo objetivo de laboratorio: recall idéntico, pero la variante “sin censura” tardó $2.8\times$ más y produjo un falso positivo adicional. (b) En el mismo sistema, corregir una única instrucción de prompt redujo de 7 a 1 los hallazgos crítico/alto que llegaban a verificación sin evidencia fresca.

rias sesiones de trabajo, no de un estudio controlado sobre múltiples sistemas. Las cifras citadas son mediciones reales de esas corridas; los nombres de objetivo, cliente y herramienta se omiten deliberadamente por confidencialidad.